

SKIjack: A High-Level Native Language for the *SKI* Calculus

N. E. Davis

September 17, 2026

Abstract

Following SKI-in-SKI, what sort of paper do you suppose we have in here?

A language paper, and the thesis is already sharper than the first paper's was at this stage, because a theorem draws it.

The result that carries it.

What it is not, yet. Without self-hosting it is a design-and-implementation paper with an artifact, honest and complete, but its claim about quotation is illustrated rather than exercised. The moment the expander is written in the surface and compiles itself, the paper gains the result that closes the argument: the compiler, the thing that performs quotation, is an ordinary program on the far side of the line it enforces. That is the second milestone in the desiderata and it is what would lift the paper from ⟨Programming⟩ or a workshop (PEPM, the Scheme workshop) to something ICFP would read. Avon is a third paper, a systems one, and should stay out of this.

Its relationship to the first paper. The first says what an interpreter does at a new symbol. This one says how to make that the normal way to write interpreters, and what a language must give up, and need not, to do so over a calculus with no self-quotation. They are a pair, and the second should cite the first for the theorem and the counts, not restate them.

If you want, when step 5 lands I'll draft the contributions list and abstract in the SKIjack repo, so the language design gets written against a paper rather than the reverse.

1 Introduction

Over a homoiconic calculus (Lisp, Nock) the metacircular tradition gets quotation for free and the language designer chooses where evaluation stages live. Over a non-homoiconic calculus that choice is taken away: Proposition 3.1 says no term can quote a live term, so the compile-time/runtime line is fixed before the language is designed, and the two operations the calculus cannot do for itself, quote and recognize, go to the compiler and the runtime respectively. Davis presented a metacircular evaluator for *SKI* which is written and evaluated in the same combinator system. This capability immediately suggests the construction of a battery of computational conveniences and abstractions within the same system; to wit, a language. We present SKIjack (pronounced “skyjack”), a closed-arm macro-like language built in and targeting the non-homoiconic *SKI* calculus as its instruction set architecture. Every named thing compiles to a closed term and every name is resolved before anything runs, so the language has no runtime environment to address: bracket abstraction has already closed what a subject-oriented calculus would have to carry. SKIjack is evaluable on any *SKI* interpreter as a virtual machine runtime.

The asymmetry that forces the staging line is older than either calculus’s programming languages, and naming it early makes the rest of the design read as consequence rather than taste. Böhm [1968] proved that two distinct $\beta\eta$ -normal forms can always be told apart from the outside: some context applies them to arguments and recovers any two results we please.¹ Jay and Given-Wilson [2011] proved that no combinator can take another apart from the inside. Both papers of this pair live in the gap between those two results. Reading a compiled term by applying it to fresh markers and seeing which one comes back is the outside; Proposition 3.1 is the inside; and a runtime jet is permitted to factorise a live term precisely because it is the Turing machine with the term on its tape, which is to say outside as well. Quotation goes to the compiler and recognition to the runtime not because that is convenient but because those are the only places the operations exist.

The `whnfF` metacircular interpreter and `UQ` unquoter function presented in Davis are the starting point for our work. That paper discussed extensions to the *SKI* calculus built on higher layers of interpreters, introducing an `Err` error type, a `Maybe` partiality type, and discussed implications such as namespace value retrieval and other innovations. By authoring a new symbol at one site, the `whnfF` approach enables extension and modification of the combinator’s semantics, permitting it to behave more like a language than a mere combinator system. This paper presents *SKIjack* and three novel contributions for *SKI* combinator research: a two-level execution model with a kernel/userspace split; and a namespace as a language feature: a resolver is a core parameter, paths are a declared type compared structurally, and the namespace literal compiles to a resolver.

2 Lineage

A language whose semantics is combinators is not a new idea. Böhm and Gross [1966] introduced the CUCH — CUrry-CHurch — a surface for writing programs whose meaning is given by combinators, and *SKIjack* is a late descendant of it. What is new here is not the layering but the constraint on it. The CUCH was free to put its conveniences wherever its designers liked; a language over a calculus that cannot quote is not, and the interest of the exercise is in what that costs and in what it does not.

Two further debts are specific. A type declaration generates its constructors and its case form by the schema Böhm and Berarducci [1985] gave for term algebras, in the Scott variant [Scott, 1962, Jansen, 2013] rather than the Church one: case analysis in place of a fold, which is what an interpreter’s step function wants and what fixes declaration order as continuation order. And the milestone that would close this paper’s argument — an expander written in the surface that compiles itself — is the oldest move in compiler construction. Böhm [1954], his 1951 dissertation at ETH Zürich, is generally credited as the first compiler written in its own language [Knuth and Pardo, 1980]; seventy-five years later the move is the same one, and here it would carry a further claim, since the compiler is the component that performs quotation and self-hosting puts it on the far side of the line it enforces.

Berarducci and Böhm [1993] give a self-interpreter of the λ -calculus having a normal form, which is the line of work the artifact this language compiles belongs to. Five of the results the present paper leans on are Böhm’s — separability, the CUCH, the term-algebra schema, the first

¹Stated for the λ -calculus and for $\beta\eta$ -distinct normal forms; see also Barendregt [1984, §10.4]. The probes this paper uses run on Scott-encoded data under weak reduction, which is not that setting, so we cite separability for the asymmetry it names and not as a completeness result for the method.

self-compiling compiler, and a self-interpreter with a normal form — which is less a coincidence than a reminder of how early these questions were asked.

3

SKIjack has one grammar and two lexicons, with a bijective token table checked at import, and the round-trip and lift laws as tests.

```
term === S | K | I | App term term
maybe === Nothing | Just term
whnfF := {
  step m = sp m nil stepS stepK stepI
  loop1 f m n2 = step m (Just m) (f n2)
  loop n m = n Nothing (loop1 loop m)
}
resS acc c0 c1 c2 = c0 acc
resK acc c0 c1 c2 = c1 acc
resI acc c0 c1 c2 = c2 acc
spApp f acc t u = f t (cons u acc)
sp m acc = m (resS acc) (resK acc) (resI acc) (spApp sp acc)
rb1 f h x xs = f (App h x) xs
rb h args = args h (rb1 rb h)
stepI1 x rest = Just (rb x rest)
stepI args = args Nothing stepI1
stepK2 x y rest = Just (rb x rest)
stepK1 x r = r Nothing (stepK2 x)
stepK args = args Nothing stepK1
stepS3 x y z rest = Just (rb (App (App x z) (App y z)) rest)
stepS2 x y r2 = r2 Nothing (stepS3 x y)
stepS1 x r = r Nothing (stepS2 x)
stepS args = args Nothing stepS1
```

Listing 1: The `whnfF` function defined in SKIjack. Pleasingly, this program produces the identical *SKI* artifact as Appendix B of [Davis](#) despite SKIjack not existing at the time of composition.

References

- Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, revised edition, 1984.
- Alessandro Berarducci and Corrado Böhm. A self-interpreter of lambda calculus having a normal form. In *Computer Science Logic (CSL '92)*, volume 702 of *Lecture Notes in Computer Science*, pages 85–99. Springer, 1993.
- Corrado Böhm. Calculatrices digitales: du déchiffrement de formules logico-mathématiques par la machine même dans la conception du programme. *Annali di Matematica Pura ed Applicata*, 37:175–217, 1954. Series 4. Originally the author’s doctoral dissertation, ETH Zürich, 1951.

- Corrado Böhm. Alcune proprietà delle forme β - η -normali nel λ -K-calcolo. Technical Report 696, Istituto per le Applicazioni del Calcolo, Rome, 1968.
- Corrado Böhm and Alessandro Berarducci. Automatic synthesis of typed λ -programs on term algebras. *Theoretical Computer Science*, 39:135–154, 1985.
- Corrado Böhm and Wolf Gross. Introduction to the CUCH. In Eduardo R. Caianiello, editor, *Automata Theory*, pages 35–65. Academic Press, New York, 1966.
- N. E. Davis. A self-interpreter for *ski* written in *ski*: Extensional semantics for new symbols. *Journal of Functional Programming*. In review.
- Jan Martin Jansen. Programming in the λ -calculus: From Church to Scott and back. In *The Beauty of Functional Code*, volume 8106 of *Lecture Notes in Computer Science*, pages 168–180. Springer, 2013.
- Barry Jay and Thomas Given-Wilson. A combinatory account of internal structure. *Journal of Symbolic Logic*, 76(3):807–826, 2011.
- Donald E. Knuth and Luis Trabb Pardo. The early development of programming languages. In N. Metropolis, J. Howlett, and Gian-Carlo Rota, editors, *A History of Computing in the Twentieth Century*, pages 197–273. Academic Press, New York, 1980.
- Dana S. Scott. A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962/63; unpublished manuscript, Stanford University, 1963, 1962.

A The artifact